

1 A cache holds many blocks of data with a tag (address), and its data.

Look for Mem[X]

HIT: $X = TAG$

Read data

Write and change data

MISS:

FUCK!!

FIND ~~X~~ and its Data

Place in a select cache line

Read or write to it.

How to quickly Read Through
Caches

Direct Mapping

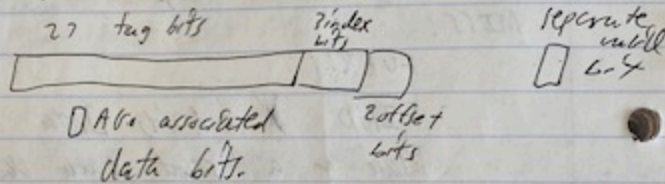
- Each memory address in CPU address space maps to a specific cache address.

• This is very simple.

- Because there is a much larger space than cache memory slots, the lines of cache have many addresses linked to each one.

1 Direct Mapping caches also use a valid bit. If the line's memory is just garbage because the cache was just flushed, If the line was not actually retrieved from memory, it is not valid.

32-bit address



1 because of locality principle, a cache is assumed to store ~~memory~~ memory in the same region, therefore making the addresses only differ in the least significant bits.

1 When looking for a particular address, the processor looks in parallel for matching index (least significant) bits. If it finds a match, it compares with rest of tag and valid bit. If correct, cache hit, otherwise nah.

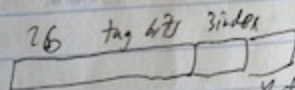
1 There are as many index bits as $\log_2$ of available cache lines. ~~offset bits is 32 minus that previous number.~~

2 Index bits are not necessarily the very least LSBs, but rather the next level up from the 2 0s, or offset bits.

Blocking and Data Size

1 Another idea for 32-bit processor: 128 data bits per line. Each line represents 4 words instead of one. Therefore not as many address bits are needed (least significant can be offset for a mux to select specific word instead), and more data stored means or much lower miss rate!

Tradeoff - worse cost for misses, but do get slightly slower, much more data retrieved.



offset
acts as mux for each word stored within block. More index would be used in larger caches.

Miss penalty increases with block size,
miss ratio is optimized between
~~block~~ at a certain block size,
and so is AMPT (around 64 bytes)

DM Cache Problems

Conflict misses - when two data
addresses that map to same cache
line are both needed consistently
for programs, making them very
likely to cause misses due to
alternating presence on the cache.

Fully Associative Cache

No indexes, only pure 32-bit tags, with
some large amounts of data per block. No set local
ity have even lower miss rates, but
have high hardware cost due to
comparing 32 bits at once for each
line.

N-Way Set Associative Cache

- A middle ground between direct mapping and FA. Can be seen as a set of smaller, parallel DM caches within one. The most significant bits are associative. The least or direct ~~sets~~ within each way.

More ways reduces conflict but increases hit time, also adds more hardware costs.

Cache Replacement Policies: How to choose a line after Cache Miss

- Locality suggests the LRU strategy:

• Replacer Least Recently Used:

Works in theory but requires $\log_2(N!)$ bits to encode order and a massive amount of hardware to find the largest number, too expensive.

- LRU is still reasonable sometimes.

Write Policies

Write Through: CPU writes are cached,
and then written to memory
immediately (CPU stalls)
- Simple, slow AF

Write-behind: CPU writes cached;
writes are buffered and executed
in the background. This is
faster but actually uses
a lot of hardware and hardware.

Write-back: CPU writes cached, but not
written to main memory until
block replacement

0 Fastest, but leads to
stale data in main; a
problem for multi-core systems.

0 You can add a dirty bit
to each cache line, enabling
writes back to main only
when bit is set, which
only happens after cache line
is written to.

ON Reference to Mem[x]: look for Tag(x)

Hit Tag(x) == Tag[i]

READ: return Data[i]

Write: change Data[i] dirty[i] = 1

MISS: Tag(x)

• replace selection

• select block to hold mem[x]

• If dirty[i] = 1 write to Mem[A], dirty[i] = 0

• Read Mem[x]

• Write: start write to mem[x], dirty = 1

CPU Pipelining

Just simply pipe 5 stages, DUH!

Not so simple.

Data Hazards

How to fix data hazards:

• Stall, Bypass, or Speculate.

• Stalling make execution kinda slow ! Might diminish pipelining benefits.

• Bypassing uses extra muxs and logic to detect hazards that gives computed values to previous stages when necessary.

~~Mux can be added to each intermediate stage~~

Mux can be added between each stage, and ~~the~~ the register file to ALU path.

~~See this~~ Some instructions cannot be resolved with just bypassing.
Ex. load instructions: always have to wait for writeback phase.
This is when stalling is necessary.
Sometimes compiler will try to prevent this, but cannot depend on them too much.

Control Hazards

Two ways of mitigation:

① Stalling:

□ Just stall until branch result is known

② Prediction:

□ Follow one of the branches.

○ If Lit, continue as is.

○ If mispred, flush pipeline and restore corrupted instructions.

0 Jump addresses should be forwarded
to ~~previous~~ earlier stage.

0 Branch prediction is a complicated
art.